

Second development changes understanding code block by block

For Page 3 of the frontend, I developed the **View Readings** screen, which represents the read side of the Cold Chain Operations Portal. The purpose of this page is to allow users to retrieve real temperature-reading data from the deployed backend rather than using temporary or mock data. I designed the page so the user can switch between two modes: **Branch Readings**, which retrieves all readings belonging to a selected branch, and **Individual Reading**, which retrieves one specific reading using the branch ID and the reading's UUID. Both operations use **TanStack Query** to handle the GET requests, caching, loading states, error states and returned data.

For the branch view, the data is displayed in a reusable database-style table instead of raw JSON. The readings are sorted with the newest first and display the Reading UUID, branch, date, time, minimum temperature, maximum temperature, status, alert-raised time and the person who recorded the reading. I separated the date and time visually even though they originate from the same `recordedAt` backend field, making the information easier to scan. I also created a reusable `StatusBadge` component so that normal readings display as `✓ OK` and excursion readings display as `⚠ EXCURSION`. This means that excursions are not identified by colour alone, improving accessibility.

The Individual Reading mode uses the second GET endpoint and allows a user to enter a branch ID and Reading UUID before selecting **Find Reading**. If the reading exists, the page shows every field returned by the backend, including the UUID, branch ID, recorded date and time, minimum and maximum temperatures, status, recorded-by information and `alertRaisedAt`. If the reading is an excursion, a prominent warning is displayed so the problem is immediately visible rather than being hidden amongst the other fields.

I also implemented separate **loading, error and empty states**. This ensures that the interface clearly communicates what is happening while a request is running, when the backend rejects a request, or when a branch exists but contains no readings. This is important because an empty result is not the same as an API error. The empty state specifically displays `"No readings recorded for this branch yet."` when the API successfully returns an empty array.

Files used for Page 3

The main files are:

```
1 app/  
2 | components/  
3 |   ReadingsTable.tsx  
4 |   StatusBadge.tsx  
5 |
```

```

6  └─ readings/
7    └─ page.tsx
8
9  lib/
10 └─ api.ts
11
12 types/
13 └─ reading.ts

```

The overall flow is:

```

1  app/readings/page.tsx
2    ↓
3  TanStack Query
4    ↓
5  lib/api.ts
6    ↓
7  Next.js API Proxy
8    ↓
9  REST API
10   ↓
11  Lambda
12   ↓
13  DynamoDB
14   ↓
15  Reading data
16   ↓
17  ReadingsTable / StatusBadge

```

This structure separates the page interface, API communication, data types and reusable UI components rather than putting everything inside one large file.

1. app/readings/page.tsx

This is the main Page 3 file. It controls the two viewing modes, the TanStack Query GET requests, the user input, and the different loading, error, empty and success states.

The file starts with:

```

1  "use client";

```

because the page needs interactive React features including `useState`, TanStack Query, input fields, buttons and the view selector.

I created a TypeScript union:

```

1  type ViewMode =
2    | "branch"
3    | "individual";

```

This means the page can only be in one of two recognised modes.

The current mode is stored using:

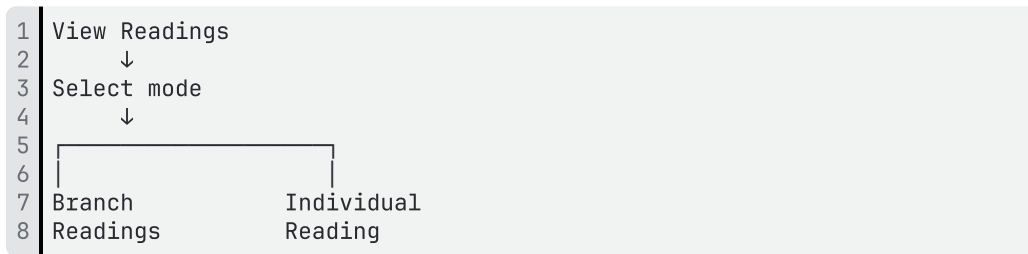
```

1  const [viewMode, setViewMode] =
2    useState<ViewMode>("branch");

```

The page begins on **Branch Readings** and changes when the user selects **Individual Reading**.

Conceptually:



This allows both GET endpoints to be used from the same main page instead of creating an unnecessary fourth page.

2. Branch Readings mode

The Branch Readings mode calls:

```
1 GET /branches/{branchId}/readings
```

using TanStack Query.

The query is structured around:

```
1 const branchQuery = useQuery({
2   queryKey: [
3     "readings",
4     branchId,
5   ],
6   queryFn: () =>
7     getBranchReadings(branchId),
8   enabled:
9     viewModel === "branch" &&
10    Boolean(branchId),
11 });
```

The `queryKey` is important because it identifies the cached data.

For example:

```
1 ["readings", "BRANCH-001"]
```

represents one branch, while:

```
1 ["readings", "BRANCH-002"]
```

represents another.

TanStack Query therefore knows that these are different sets of data.

The `queryFn` calls:

```
1 | getBranchReadings(branchId)
```

from `lib/api.ts` rather than placing the complete `fetch()` request inside the page itself.

The `enabled` condition prevents the branch query from running unnecessarily when the page is in Individual Reading mode.

3. Branch loading state

While the request is being processed, I check:

```
1 | branchQuery.isLoading
```

and display:

```
1 | Loading temperature readings...
```

This gives the user immediate feedback instead of displaying an empty page while the request is waiting for the backend.

4. Branch error state

If the request fails:

```
1 | branchQuery.isError
```

becomes true.

The page then displays an error message instead of attempting to display the table.

Where possible, the real error returned through the API layer is shown to the user, which is more useful than only displaying a generic `"Something went wrong"` message.

5. Empty state

A successful API response can still contain no readings.

I check:

```
1 | branchQuery.data?.length === 0
```

and display:

```
1 | No readings recorded for this branch yet.
```

This is deliberately handled separately from an error.

```
1 | ERROR
2 | ↓
3 | Request failed
```

```
4
5 EMPTY
6 ↓
7 Request succeeded
8 ↓
9 0 readings returned
```

That distinction gives the user a much clearer understanding of the result.

6. Successful branch response

When readings are successfully returned, I pass them into:

```
1 <ReadingsTable
2   readings={branchQuery.data}
3 />
```

This means `page.tsx` does not need to contain all the table-rendering code itself.

Instead:

```
1 page.tsx
2   ↓
3 gets the data
4   ↓
5 ReadingsTable.tsx
6   ↓
7 displays the data
```

This makes the frontend easier to understand and maintain.

7. ReadingsTable.tsx

`ReadingsTable.tsx` is a reusable component responsible for displaying the branch readings in a structured table.

It receives:

```
1 type ReadingsTableProps = {
2   readings: Reading[];
3 };
```

The `Reading[]` type tells TypeScript that the component expects an array of reading objects.

Sorting newest first

The ticket requires the newest readings to appear first.

I therefore use:

```
1 const sortedReadings =
2   [...readings].sort(
3     (a, b) =>
4       new Date(
```

```

5         b.recordedAt
6     ).getTime() -
7     new Date(
8         a.recordedAt
9     ).getTime()
10 );

```

This compares the `recordedAt` timestamps and places the most recent timestamp first.

I use:

```

1 | [...readings]

```

rather than sorting the original `readings` array directly.

The spread syntax creates a copy first, which means I do not directly modify the cached data supplied by TanStack Query.

8. Date and time formatting

The backend stores one value:

```

1 | recordedAt

```

such as:

```

1 | 2026-09-09T13:56:27.743Z

```

For the user interface I format it into separate:

```

1 | Date
2 | Time

```

columns.

The table uses functions such as:

```

1 | formatDate()

```

and:

```

1 | formatTime()

```

with British formatting.

This makes the data easier to read while leaving the original backend timestamp unchanged.

9. Alert Raised At

The table also displays:

```

1 | Alert Raised At

```

using the optional `alertRaisedAt` field.

There are three possible states:

```
1 | alertRaisedAt exists
2 | → show the timestamp
3 |
4 | excursion but no timestamp yet
5 | → Pending
6 |
7 | normal reading
8 | → -
```

This avoids leaving an unexplained blank field and clearly distinguishes a normal reading from an excursion whose alert information may not yet be available.

10. Table columns

The final table contains:

```
1 | Reading UUID
2 | Branch
3 | Date
4 | Time
5 | Min °C
6 | Max °C
7 | Status
8 | Alert Raised At
9 | Recorded By
```

The UUID is displayed separately because `readingId` is the unique identifier for each reading.

The date and time are also displayed separately for readability even though both originate from `recordedAt` .

11. Creating table rows

The component uses:

```
1 | sortedReadings.map(
2 |   (reading) => (
```

to create one table row for every reading returned by the backend.

For example:

```
1 | 5 Reading objects
2 |   ↓
3 |   map()
4 |   ↓
5 | 5 table rows
```

Each row uses:

```
1 | key={reading.readingId}
```

because the UUID is unique for every reading.

12. Excursion highlighting

When:

```
1 | reading.status ===  
2 | "excursion"
```

the row receives additional warning styling.

However, I deliberately do not rely only on a red background.

The status also displays:

```
1 | ⚠ EXCURSION
```

so the meaning remains clear even to a user who cannot distinguish the warning colour.

13. StatusBadge.tsx

I created `StatusBadge.tsx` so that the same status design can be reused throughout the frontend.

It receives:

```
1 | type StatusBadgeProps = {  
2 |   status: ReadingStatus;  
3 | };
```

where:

```
1 | type ReadingStatus =  
2 |   "ok" | "excursion";
```

TypeScript therefore prevents unexpected status values being passed into the component.

For an excursion it displays:

```
1 | ⚠ EXCURSION
```

and for a normal reading:

```
1 | ✓ OK
```

The combination of text, symbol and colour means the application does not rely on colour alone to communicate status.

14. Individual Reading mode

The second mode allows the user to retrieve one specific reading.

It uses:

```
1 GET /branches/{branchId}/readings/{readingId}
```

The user supplies:

```
1 Branch ID
2 Reading UUID
```

and presses:

```
1 Find Reading
```

The UUID is stored using React state:

```
1 const [
2   readingId,
3   setReadingId,
4 ] = useState("");
```

15. Individual TanStack Query

The individual reading has its own query:

```
1 const individualQuery =
2   useQuery({
```

with a key based on:

```
1 reading
2 branch ID
3 reading UUID
```

For example:

```
1 [
2   "reading",
3   "BRANCH-001",
4   "ba9de34f-..."
5 ]
```

This allows TanStack Query to cache individual readings independently.

The query eventually calls:

```
1 getReadingById(
2   branchId,
3   readingId
4 )
```

from `lib/api.ts`.

16. Manual individual search

Unlike the branch query, the individual query uses:

```
1 | enabled: false
```

so it does not immediately send a GET request when the page loads.

The request is triggered through the **Find Reading** interaction.

The button is also disabled if either the branch ID or UUID is blank.

This prevents incomplete requests from being sent unnecessarily.

17. Individual loading and error states

While searching for an individual reading, the page uses:

```
1 | individualQuery.isFetching
```

to display:

```
1 | Loading reading...
```

If the request fails:

```
1 | individualQuery.isError
```

is used to show the error.

For example, if the UUID does not exist, the backend can return a `404`, and the interface can communicate that failure to the user rather than showing an empty details card.

18. Individual reading details

When:

```
1 | individualQuery.data
```

exists, the page displays the reading in a dedicated details card.

The card contains:

```
1 | Reading UUID
2 | Branch ID
3 | Recorded At
4 | Recorded By
5 | Minimum Temperature
6 | Maximum Temperature
7 | Status
8 | Alert Raised At
```

This means the individual mode shows every field available from the frontend `Reading` structure.

19. Excursion warning

If the individual reading has:

```
1 | status = excursion
```

the page displays a prominent warning:

```
1 | △ Excursion detected
2 |
3 | This reading is outside the
4 | permitted temperature range.
```

The normal `StatusBadge` is still displayed as well.

This makes an excursion immediately noticeable instead of requiring the user to search through the details for the status field.

20. Showing `alertRaisedAt`

The individual details also display:

```
1 | Alert Raised At
```

when the backend returns the value.

This gives the user visibility of when the excursion alert was processed.

If no timestamp exists, the interface displays:

```
1 | -
```

rather than an unexplained blank space.

21. `lib/api.ts`

Page 3 uses two API functions:

```
1 | getBranchReadings()
2 | getReadingById()
```

These represent the two GET operations.

This separation means `app/readings/page.tsx` does not need to contain:

```
1 | API URL
2 | API key
3 | proxy implementation
```

It simply calls the appropriate API function.

The lower-level API details remain inside `lib/api.ts`, keeping the page focused on interface behaviour.

22. types/reading.ts

Page 3 also uses the shared `Reading` TypeScript type.

The structure contains:

```
1 readingId
2 branchId
3 recordedAt
4 minTempC
5 maxTempC
6 status
7 recordedBy
8 alertRaisedAt
```

This allows TypeScript to understand the structure of the data returned by the backend.

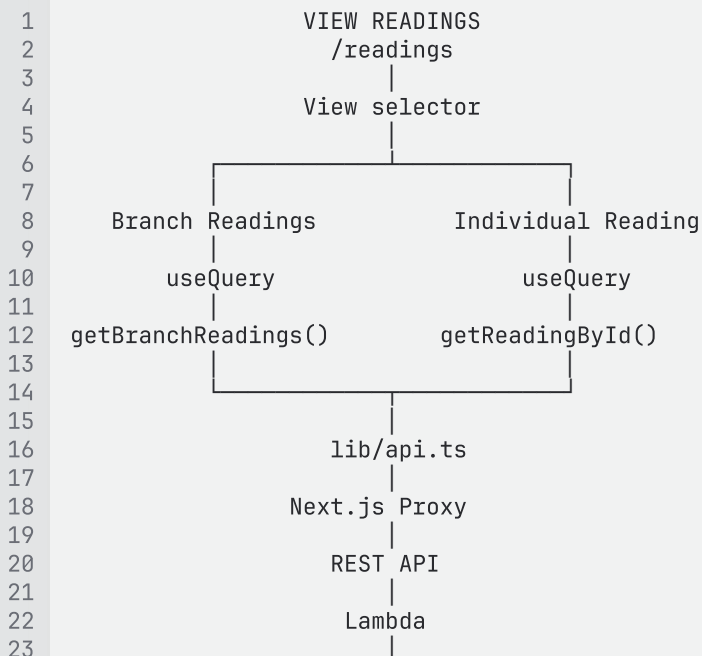
For example, TypeScript knows:

```
1 minTempC: number
```

rather than treating it as an unknown value.

Final Page 3 architecture

The completed page works like this:



This makes Page 3 the dedicated **read/view side** of the Cold Chain Operations Portal.

Page 3 completed requirements

- ☒ Real deployed backend data
- ☒ TanStack Query used for GET requests
- ☒ GET all readings for a branch
- ☒ GET individual reading by UUID
- ☒ Branch input
- ☒ Branch/Individual view selector
- ☒ Database-style readings table
- ☒ Newest readings displayed first
- ☒ Reading UUID displayed separately
- ☒ Branch displayed
- ☒ Date and time displayed separately
- ☒ Minimum and maximum temperature displayed
- ☒ Status displayed
- ☒ Alert Raised At displayed
- ☒ Recorded By displayed
- ☒ Individual reading detail
- ☒ Prominent excursion warning
- ☒ ⚠ EXCURSION text and icon
- ☒ Does not rely on colour alone
- ☒ Loading state
- ☒ Error state
- ☒ Empty state
- ☒ "No readings recorded for this branch yet."
- ☒ Reusable TypeScript components
- ☒ Shared Reading type
- ☒ Real API rather than mock data